

CANDY DIALOGUE ENGINE

Engine Structure

1.1.0

WARNING: Candy_Engine.gd is overwritten during updates. Always document any modifications you make to it, so that you can re-add them later.

This guide is meant to provide you with a mental framework of how the engine code's logic functions, should you want to modify the code.

Engine Core

The engine's core is the **Candy_Engine.gd** script.

This script handles the dialogue processing logic from start to finish, occasionally deferring to other scripts such as UI Element scripts or hook functions.

Step 0: Loading Dialogue

Before a dialogue can be run, it must be loaded in the scripts running_dialogue dictionary. The **load_scripted_dialogue()** function can be used to load a dialogue from a .txt file, but other options can be used (see the **Starting Dialogues** guide for more information about this).

Step 1: Starting Dialogue

The actual process of running dialogues begins with the **start_dialogue()** function, which developers must setup their game to call as needed.

This function is provided the Conversation, Block and Line to start at, makes the dialogue UI visible, changes the mouse_mode as configured by the developer, and calls the **run_dialogue()** function to move on to the next step.

Step 2: Line Iteration

The **run_dialogue()** function's main role is simply to iterate through Lines in a given Block, to have them processed by the rest of the code by calling the **process_lines()** function.

Step 3: Line Processing Part 1

The `process_lines()` function essentially receives a Line from the `run_dialogue()` function and begins processing it.

First, it determines whether the Line is a Command or a Spoken Line.

If the Line is a Command, `process_lines()` calls the `commands()` function.

If the Line is a Spoken Line, it calls the `display_lines()` function.

Step 4-A: Commands

The `commands()` function uses a match statement to run the appropriate logic depending on what kind of command a Line is.

If you think of the engine logic as a tree, the previous functions form the trunk, while commands are branches.

This design allows adding any command you could imagine, by simply adding a match case for it. Commands don't modify the core logic, and they are independent of one-another.

Commands may further call helper functions inside `Candy_Engine.gd`, or defer to the code of dialogue UI Elements.

Ultimately, once Commands have done their work, they return to the `process_lines()` function.

Step 4-B: Spoken Lines

When the `process_lines()` function processes a Spoken Line, it doesn't immediately call the `display_lines()` function.

Instead, it first partially prepares the line, by doing the following:

1. It resolves Dispositions, to check whether the line should run or be skipped (see the `Dispositions` guide).
2. It determines which variant to use for the spoken text (see the `Variants & Translations` guide).
3. It runs the Variable Substitution logic, to replace variables references in the spoken text with their values (see the `Variables` guide).
4. It runs the Word Substitution logic (see the `Word Substitution` guide).
5. It runs the LLM logic, if your game is setup to query an LLM to generate spoken text.

After doing all the steps above, the `process_lines()` function finally calls `display_lines()` and passes the relevant data to it.

Once the `display_lines()` function is called, it also does its own work processing data about the line before displaying it:

1. It resolves which voice file to use, if one is specified for the line.
2. It resolves the formatting of the speaker's name, using BBCode tags, if this is configured.
3. If Visual Novel Mode is enabled, and the dialogue mode is set to "Bubbles", it switches to "VN Bubbles" for compatibility.

Finally, the `display_lines()` function uses a match statement to run the correct logic for the dialogue mode that should be used.

Each dialogue mode can have its own unique logic (similar to how commands differ from one another), but dialogue modes also tend to have some similarities (note that "Voice" mode is a major exception as it doesn't display any text).

For instance, the built-in dialogue modes include logic to set the color of the text as configured (see the `Text Color` guide), and query a TTS system if this option is enabled (see the `TTS` guide).

Dialogue modes then display the text on screen, either instantly or progressively (typewriter effect), and respond to player input to speed up, slow down or skip progressive writing.

Finally, dialogue modes await player input before letting the engine process the next line (unless configured to automatically move on after a set amount of time).

Step 5: Line Processing Part 2

The `commands()` and the `display_lines()` functions always return to the `process_lines()` function that called them.

The `process_lines()` function tidies things up before the engine moves on to the next line: it decrements the number of elapsed lines for paused media players (and makes them resume if appropriate), it logs data to the history log, and a few other cleanup operations.

It then returns to the `run_dialogue()` function that called it.

Step 6: Continuing

The `run_dialogue()` function now does either of two things: if there are still Lines to process in the Block, it repeats from Step 2, processing the next Line.

If there are no more Lines in the Block, or if a `$Return` command was last processed, it attempts to return to the previous Block if there is one. There can only be a previous Block if the current Block was reached by a `$Bridge` command (see the `Transitions` guide for details).

If there are no previous Blocks, or if an `$End` command was last processed, `run_dialogue()` ends the entire dialogue by calling the `end_dialogue()` function.

Step 7: Ending Dialogue

The `end_dialogue()` function does several things to end dialogue:

1. It calls the `change_game_state()` function in `Candy_Functions.gd` to restore game state according to the logic you defined.
2. It calls the `clear_ui()` function to clean up the dialogue UI.
3. It changes the `Mouse_Mode` as configured.
4. It stops all media that are being played by media players listed in the `media_players_locations` dictionary (`Candy_Engine.gd`).

The `clear_ui()` function does the following:

1. Erases the spoken text, speaker name, and hides all speech UI Elements (dialogue box, speech bubbles, chat box, and subtitles).
2. Clears portraits.
3. Clears visual novel busts, unless configured to keep busts persistent.
4. Clears backgrounds, unless configured to keep background layers persistent.

At this point, all code and logic related to the dialogue should cease running.

Special Cases

Bridged (Nested) Blocks

There is one special case you should be aware of: nested Blocks.

Nested Blocks occur when a `§Bridge` command is processed: the `§Bridge` command is used to transition to a new Block momentarily (or another Line in the same Block).

To achieve this, the `§Bridge` command calls and awaits on a new instance of the `run_dialogue()` function. This results in multiple `run_dialogue()` functions being 'opened' in parallel, although only the last one is ever working: the others are paused, waiting for subsequent ones to finish.

These new `run_dialogue()` functions simply do their thing, iterating through the lines in the Block they're assigned and having the rest of the code process them. Once they reach the end of their Block, unless they reach an `§End` command, they terminate and let the previous `run_dialogue()` function resume.

If any of these `run_dialogue()` functions processes an `§End` command, they end the dialogue by starting a chain reaction of self-termination: the latest such function terminates, and sends a signal to the one before it to terminate as well.

The signal propagates up the chain, until the very first `run_dialogue()` function is reached. This original function then calls the `end_dialogue()` function.

Jumped Blocks

The `§Jump` command works differently from the `§Bridge` command. Like `§Bridge`, `§Jump` transitions to another Block (or Line) but this transition is meant to be permanent, with no option to return to the Block that the `§Jump` command originated from.

Instead of directly calling its own `run_dialogue()` function, the `§Jump` command calls `jump_start_dialogue()`, and ends the current dialogue.

The `jump_start_dialogue()` function starts a new dialogue at the Conversation, Block and Line provided in the `§Jump` command. This is similar to `start_dialogue()`, but `jump_start_dialogue()` contains special logic to ensure dialogue continuity. To be precise, it awaits for the `greenlight` to be reset to 'true' before calling the new `run_dialogue()`, otherwise the new `run_dialogue()` would immediately self-terminate.

If you're planning to dive deep into the engine code, we can't promise that the `§Bridge` and `§Jump` logic will be simple to understand. However, unless you're trying to modify that part of the engine logic, or trying to create commands that work similarly to `§Jump` and `§Bridge`, you shouldn't have any need to concern yourself with this.